

LM: prob distribution over a sequence of tokens.

$$\frac{P(I \text{ am going to school})}{P(I \text{ going to am})} = \frac{0.99}{0.001} = 0.99$$

$$\frac{P(I \text{ eat dum})}{P(I \text{ eat dum})} = 1.000$$

Statistical Language Model

Next word prediction model

$P(I \text{ am going to school} | \text{field, pen, notes, pen})$

$$P(a \text{ b c}) = P(a \text{ b c}) = 0.0001$$

Chain Rule

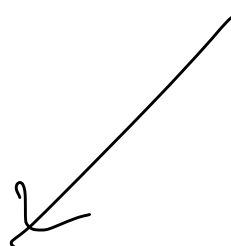
$$P(I \text{ eat rice}) = P(I) P(eat | I) P(rice | I \text{ eat})$$

$$P(A B C) = P(A) P(B | A) P(C | A B)$$

Word Prediction

- Guess the next word...

... I notice three guys standing on the ???



- There are many sources of knowledge that could be helpful, including world knowledge.
- But we can do pretty well by simply looking at the preceding words and keeping track of simple counts.

Word Prediction

- Formalize this task using *N*-gram models.
- *N*-grams are token sequences of length *N*.
- Given *N*-grams counts, we can guess likely next words in a sequence.

Probabilistic Language Models

The goal: assign a probability to a sentence

- Machine Translation:
 - $P(\text{high winds tonite}) > P(\text{large winds tonite})$
- Spelling Correction
 - The office is about fifteen **minuets** from my house
 - $P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$
- Speech Recognition
 - $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$
- + Summarization, question-answering, etc., etc.!!

la
large h

Probabilistic Language Modeling

- Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

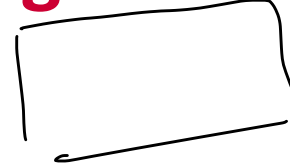
- Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- A model that computes either of these:

$P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$ is called a **language model**.

- Better: **the grammar** But **language model** or **LM** is standard



How to compute $P(W)$

- How to compute this joint probability:

$P(\text{its water is so transparent that the}) =$

$$\frac{\text{Count}(\text{its water is so transparent that the})}{\text{Count}(\text{its water is so transparent that})}$$

$P(\text{its, water, is, so, transparent, that})$

- Intuition:** let's rely on the Chain Rule of Probability

$$P(I \text{ eat rice}) = P(I) P(\text{eat} | I) P(\text{rice} | I \text{ eat})$$

$\frac{\text{Count}(I)}{\text{Count}(\# \text{ food})}$

$\frac{\text{Count}(I \text{ eat})}{\text{Count}(I)}$

$\frac{\text{Count}(I \text{ eat rice})}{\text{Count}(I \text{ eat})}$

uniform
 high
 4-20

bigram

The Chain Rule: General

- The definition of conditional probabilities

$$P(A | B) = P(A, B) / P(B)$$

Rewriting: $P(A, B) = P(A | B) P(B)$

- More variables:

$$P(A, B, C, D) = P(A)P(B | A)P(C | A, B)P(D | A, B, C)$$

- The Chain Rule in General

$$\underline{P(x_1, x_2, x_3, \dots, x_n)} = \underline{P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2) \dots P(x_n | x_1, \dots, x_{n-1})}$$

The Chain Rule: joint probability in sentence

$$P(\underline{w_1 w_2 \dots w_n}) = \prod_i P(\underline{w_i \mid w_1 w_2 \dots w_{i-1}})$$

P(“its water is so transparent”) =

P(its) × P(water|its) × P(is|its water) ×

P(so|its water is) × P(transparent|its water is so)

How to estimate these probabilities

- Could we just count and divide?

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{\textit{Count}(\text{its water is so transparent that the})}{\textit{Count}(\text{its water is so transparent that})}$$

- No! Too many possible sentences!
- We'll never see enough data for estimating these

Markov Assumption

- Simplifying assumption:

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$$

- or maybe

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{transparent that})$$

trigram
Bigram



Andrei Markov

~ P(the)
in (that the)
in (that)

Markov Assumption

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i \mid w_{i-k} \dots w_{i-1})$$

- In other words, we approximate each component in the product

$$P(w_i \mid w_1 w_2 \dots w_{i-1}) \approx P(w_i \mid w_{i-k} \dots w_{i-1})$$

Simplest case: Unigram model

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from a unigram model

fifth, an, of, futures, the, an, incorporated, a, a,
the, inflation, most, dollars, quarter, in, is, mass

thrift, did, eighty, said, hard, 'm, july, bullish

that, or, limited, the

Bigram model

$$P(w_i | w_{i-2} w_{i-1})$$

- Condition on the previous word:

$$P(\underline{w_i} | \underline{w_1 w_2 \dots w_{i-1}}) \approx P(w_i | \underline{w_{i-1}})$$

texaco, rose, one, in, this, issue, is, pursuing,
growth, in, a, boiler, house, said, mr., gurria, mexico,
's, motion, control, proposal, without, permission,
from, five, hundred, fifty, five, yen

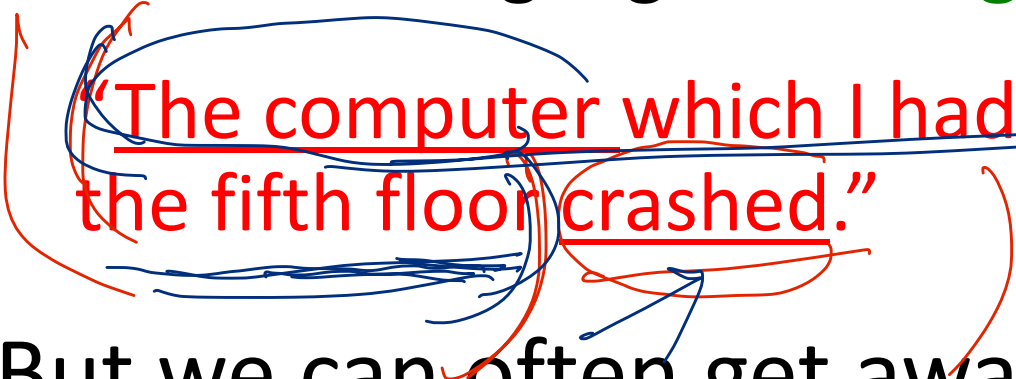
outside, new, car, parking, lot, of, the, agreement,
reached

this, would, be, a, record, november

N-gram models

- We can extend to trigrams, 4-grams, 5-grams
- In general this is an insufficient model of language
 - because language has **long-distance dependencies**:

"The computer which I had just put into the machine room on the fifth floor crashed."



- But we can often get away with N-gram models

Estimating N-gram Probabilities

Estimating bigram probabilities

- The Maximum Likelihood Estimate (MLE)

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

An example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

begin

unigram

$\langle s \rangle$ I am Sam $\langle /s \rangle$

$\langle s \rangle$ Sam I am $\langle /s \rangle$

$\langle s \rangle$ I do not like green eggs and ham $\langle /s \rangle$

$P(\text{Sam} | \langle s \rangle) = \frac{c(\langle s \rangle \text{ Sam})}{c(\langle s \rangle)}$

$\frac{1}{3}$

$$P(\text{I} | \langle s \rangle) = \frac{2}{3} = .67$$

$$P(\langle /s \rangle | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \langle s \rangle) = \frac{1}{3} = .33$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

I do and ham

$$= P(\text{I}) \times P(\text{do} | \text{I}) \times P(\text{and} | \text{do})$$

$$P(\text{am} | \text{I}) = \frac{1}{2}$$

P(do)

$$= P(I) \times \cancel{P(d_o/I)} \times \cancel{P(d_i/I, d_o)} \times P(h/I, d_o)$$

More examples: Berkeley Restaurant Project sentences

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

Type 2

Raw bigram counts (absolute measure)

- Out of 9222 sentences

Bigram table

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

I

Raw bigram probabilities (relative measure)

derived the (a)

- Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Result:

$P(\text{word} | i)$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

$P(\text{want} | i)$

$$P(I \text{ for } L) = P(I) P(L|I) \approx 0$$

Bigram estimates of sentence probabilities

$$P(<s> \text{ I want english food } </s>) =$$

$$P(I | <s>)$$

$$\times P(\text{want} | I)$$

$$\times P(\text{english} | \text{want})$$

$$\times P(\text{food} | \text{english})$$

$$\times P(</s> | \text{food})$$

$$= .000031$$

Smoothing

What kinds of knowledge?

- $P(\text{english} | \text{want}) = .0011$ world
- $P(\text{chinese} | \text{want}) = .0065$
- $P(\text{to} | \text{want}) = .66$ grammar
- $P(\text{eat} | \text{to}) = .28$
- $P(\text{food} | \text{to}) = 0$ grammar (contingent zero)
- $P(\text{want} | \text{spend}) = 0$ grammar (structural zero)
- $P(i | \langle s \rangle) = .25$

The **right to food**, and its non variations, is a human right protecting the right for people to feed themselves in dignity

Practical Issues

- We do everything in log space
 - Avoid underflow: multiplying extremely small numbers
 - Adding is faster than multiplying

$$p_1 \times p_2 \times p_3 \times p_4 \Rightarrow \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Generalization and zeros

The Wall Street Journal is not Shakespeare

Unigram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

Bigram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

Trigram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

The perils of overfitting

- N-grams only work well for word prediction if the test corpus looks like the training corpus
 - In real life, it often doesn't
 - We need to train robust models that generalize!
 - One kind of generalization: Zeros!
 - Things that don't ever occur in the training set
 - But occur in the test set

	eat	chinese	food	lunch	spend			
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0072	0	0.0017	0.01	0.0063	0.0045	0.004	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0043	0	0	0	0.52	0.0063	0	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Zeros

- Training set:
 - ... denied the allegations
 - ... denied the reports
 - ... denied the claims
 - ... denied the request
- Test set
 - ... denied the offer
 - ... denied the loan

$$P(\text{"offer"} \mid \text{denied the}) = 0$$

Zero probability bigrams

- Bigrams with zero probability
 - mean that we will assign 0 probability to the test set!
- And hence we cannot compute perplexity (can't divide by 0)!
- Zero mitigation
 - Various **smoothing** techniques

Basic Smoothing: Interpolation and Back-off

The intuition of smoothing

- When we have sparse statistics:

$P(w \mid \text{denied the})$

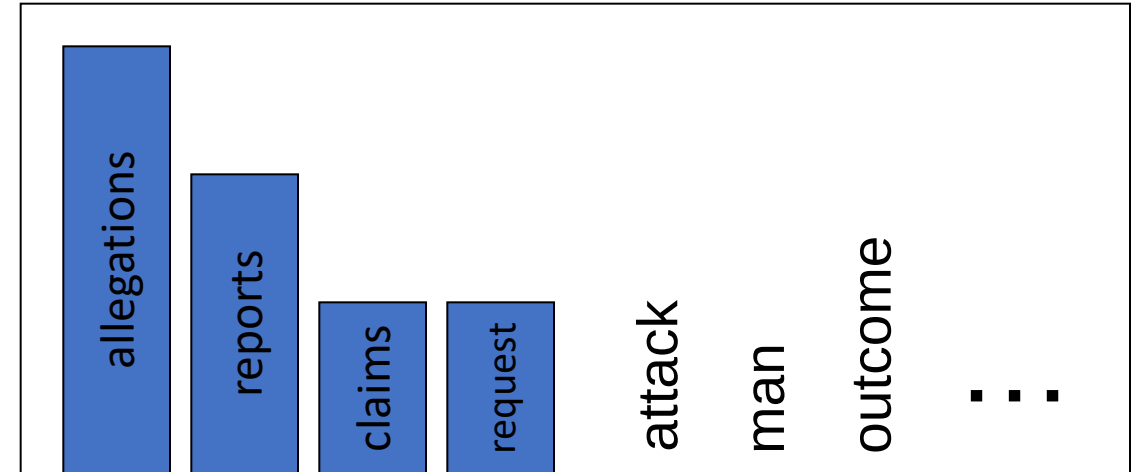
3 allegations

2 reports

1 claims

1 request

7 total



- Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

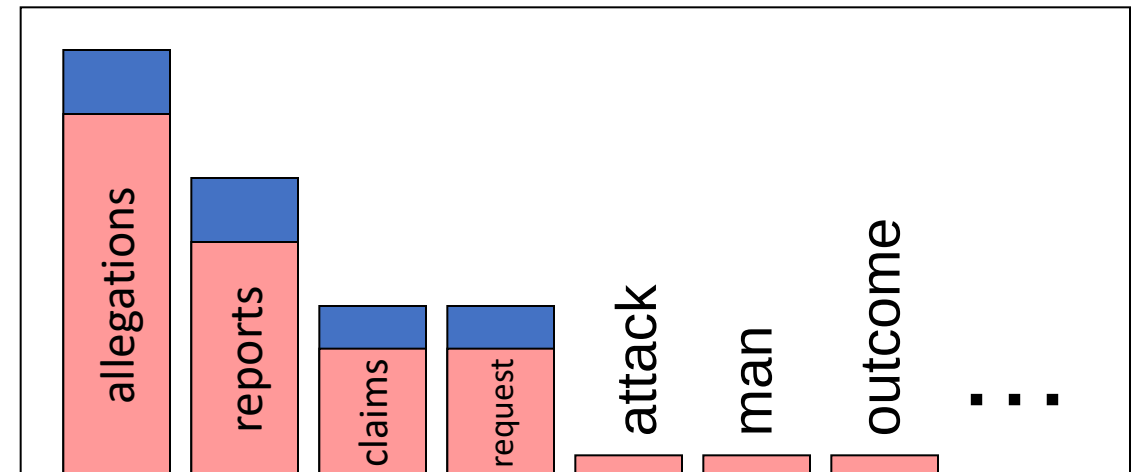
1.5 reports

0.5 claims

0.5 request

2 other

7 total



Add-one estimation

- Also called Laplace smoothing
- Pretend we saw each word one more time than we did
- Just add one to all the counts!

- MLE estimate:
$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- Add-1 estimate:
$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

Maximum Likelihood Estimates

- The maximum likelihood estimate
 - of some parameter of a model M from a training set T
 - maximizes the likelihood of the training set T given the model M
- Suppose the word “bagel” occurs 400 times in a corpus of a million words
- What is the probability that a random word from some other text will be “bagel”?
- MLE estimate is $400/1,000,000 = .0004$
- This may be a bad estimate for some other corpus
 - But it is the **estimate** that makes it **most likely** that “bagel” will occur 400 times in a million word corpus.

Berkeley Restaurant Corpus: Laplace smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Laplace-smoothed bigrams

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Reconstituted counts

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n) + 1] \times C(w_{n-1})}{C(w_{n-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Compare with raw bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Add-1 estimation is a blunt instrument

- So add-1 isn't used for N-grams:
 - We'll see better methods
- But add-1 is used to smooth other NLP models
 - For text classification
 - In domains where the number of zeros isn't so huge.

Backoff and Interpolation

- Sometimes it helps to use **less** context
 - Condition on less context for contexts you haven't learned much about
- **Backoff:**
 - use trigram if you have good evidence,
 - otherwise bigram, otherwise unigram
- **Interpolation:**
 - mix unigram, bigram, trigram
- Interpolation works better

Linear Interpolation

- Simple interpolation

$$\hat{P}(w_n | w_{n-2} w_{n-1}) = \lambda_1 P(w_n | w_{n-2} w_{n-1}) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n)$$
$$\sum_i \lambda_i = 1$$

- Lambdas conditional on context:

$$\hat{P}(w_n | w_{n-2} w_{n-1}) = \lambda_1(w_{n-2}^{n-1}) P(w_n | w_{n-2} w_{n-1}) + \lambda_2(w_{n-2}^{n-1}) P(w_n | w_{n-1}) + \lambda_3(w_{n-2}^{n-1}) P(w_n)$$

Unknown words: Open versus closed vocabulary tasks

- If we know all the words in advanced
 - Vocabulary V is fixed
 - Closed vocabulary task
- Often we don't know this
 - **Out Of Vocabulary** = OOV words
 - Open vocabulary task
- Instead: create an unknown word token <UNK>
 - Training of <UNK> probabilities
 - Create a fixed lexicon L of size V
 - At text normalization phase, any training word not in L changed to <UNK>
 - Now we train its probabilities like a normal word
 - At decoding time
 - If text input: Use UNK probabilities for any word not in training